

Claude Code CLI x WSL 環境

Claude Code

入門實作課

從安裝到工作流，一次跑完環境、指令、SubAgent 與
兩個 Demo

TODAY

今天結束， 你會帶走 →

- ✓ 在 WSL 裝好完整開發環境
- ✓ Claude Code CLI 基本操作
- ✓ 搞懂 .claude/ 目錄與 CLAUDE.md
- ✓ 用工作流跑出 Flask API + Dashboard
- ✓ SubAgent 怎麼幫你平行處理

CHAPTER 01

環境準備

完成開發環境建置

安裝開發工具

git

版本控制

nvm + node

前端工具鏈需要

UV

Python 套件管理（比 pip 快很多）

重點：直接叫 Claude Code 幫你裝

prompt-安裝環境

初始化開發環境：

- 安裝 Git
- 安裝 nvm
- 使用 nvm 安裝最新 Node.js LTS 並設為預設版本
- 安裝 uv

若工具已存在則跳過，使用官方推薦安裝方式

terminal-驗證安裝

```
$ source ~/.bashrc # 重新載入環境變數
$ echo "Git: $(git --version)" # 確認各工具版本
$ echo "NVM: $(nvm -v)" # Node 版本管理器
$ echo "Node: $(node -v)" # JavaScript 執行環境
$ echo "UV: $(uv --version)" # Python 套件管理器
```

下載教材 + WSL 檔案系統

terminal - 下載教材

```
$ cd /mnt/d # 切換到 D 槽
$ git clone https://github.com/burinexovo/claude-code-course.git # 下載課程教材
$ mkdir my-lab # 建立練習用隔離資料夾
$ cd my-lab # 進入練習資料夾
```

Linux 根目錄 /

home/	使用者家目錄
mnt/	Windows 磁碟 (c/d)
etc/	系統設定檔
usr/	已安裝的程式
var/	log、快取
tmp/	暫存檔

WSL 路徑 ↔ Windows

/home/username/
Linux 家目錄（主工作區）

/mnt/c/
Windows C 槽

/mnt/d/
Windows D 槽

 檔案總管網址列輸入 \\wsl\$ 即可從 Windows 開啟

CHAPTER 02

四種操作模式

你可以怎麼指揮 Claude Code？

四種操作模式

切換：Ctrl + Shift

Normal mode	問答模式，若要執行指令、寫檔操作都會先問你同不同意	不確定它會做什麼時
Plan mode	先輸出完整計畫讓你審核，確認後才動手	大改動前
Accept edits mode	檔案修改自動接受，shell 指令還是會問	信任任務內容時
Auto mode	全自動，什麼都不問	跑確定的重複任務

CHAPTER 03

特殊符號 / ! @

三個符號，大幅擴展對話能力

SECTION 03 · 特殊符號

特殊符號 / ! @



呼叫 slash commands

```
> /exit    離開 CLI  
> /clear   清空 session，開新對話  
> /skills  列出所有可用 Skills
```



直接執行 shell 指令

```
> !ls -la  
> !git status && git diff
```



把指定檔案加進對話

```
> @note.txt 幫我重整筆記  
> @package.json 幫我更新依賴說明
```



WSL 無法直接貼圖片 但可用 @ 路徑帶入圖片讓 Claude 分析。

CHAPTER 04

.claude/ 資料夾結構

Claude Code 的設定都在這裡

.claude/ 資料夾結構

your-project /

• CLAUDE.md

• .mcp.json

• .worktreeinclude

• **.claude/**

• settings.json

• settings.local.json

• rules/

• skills/

• commands/

• output-styles/

• agents/

• workflows/

• agent-memory/

CLAUDE.md

每次 session 自動讀取的專案指示

settings.json

權限、hooks、設定

rules/

主題式指示，可依路徑條件載入

skills/

可用 /name 呼叫的可重用 prompt

commands/

單一 .md 檔指令，輕量版 skills/

	commands/	skills/
結構	單一 .md 檔	資料夾 + SKILL.md
呼叫	/name	/name (相同)
附帶檔案	✗	✓ 可 bundle
現狀	舊機制，仍支援	官方推薦

agents/

有獨立 context 的自訂 SubAgent

完整 .claude/ 目錄說明

code.claude.com/docs/zh-TW/claude-directory

CHAPTER 05

基本指令

+ TODO List 實作

常用指令一覽

輸入技巧

\ + Enter

換行，不送出

Ctrl + Enter

換行，不送出

初始化

/init

自動生成 CLAUDE.md，新專案第一件事

/memory

開啟 memory 檔案編輯

/status

查看版本、模型、帳號狀態

上下文管理

/clear

清空對話開新 session

/compact

壓縮摘要釋放空間

/context

視覺化 context 用量

/rewind

回到上一個 checkpoint（雙 ESC 也可以）

模型與運算

/model

切換模型

/effort

調整思考強度（low / medium / high / max）

開發輔助

/diff

互動式查看 uncommitted 變更

/goal

設定目標，Claude 持續工作到達成

費用追蹤

/usage

查看花費、用量統計

常用指令一覽（續）

Plugin & Skill	離開	Session 管理
/plugin 管理 Plugin /skills 列出所有可用 Skills /reload-skills 裝完新 Skill 後重新載入 /reload-plugins 同上，套用於 Plugin	/exit 離開 CLI	claude --continue (-c) 續接上次 session ※ terminal claude --resume (-r) 列出歷史 session 選接 ※ terminal /resume 在對話中切換 session

TODO List 成果

Todo List

新增全部未完成已完成

- | | | |
|-------------------------------------|---------|---|
| ☐ | 買牛奶和雞蛋 | × |
| ☐ | 回覆媽媽的訊息 | × |
| ☒ | 預約牙醫門診 | × |
| ☐ | 繳水電費 | × |
| ☐ | 去健身房運動 | × |

4 項未完成

清除已完成

SECTION 05 · 視覺品質對比

結合 frontend-design skill 設計

`/frontend-design` 告訴 Claude 你想要的風格 → 生成高品質 UI

預設輸出

Todo List

新增待辦事項... 新增

全部 未完成 已完成

- ☐ 買牛奶和雞蛋
- ☐ 回覆媽媽的訊息
- ☒ 預約牙醫門診
- ☐ 繳水電費
- ☐ 去健身房運動

4 項未完成 清除已完成

陽春、generic



frontend-design

My Todo List

ORDER #344

今天要做什麼... ADD

PRIORITY DUE 年/月/日

ALL OPEN DONE

- ☒ 預約牙醫門診
- ☐ 繳水電費
- ☒ 買牛奶和雞蛋
- ☐ 回覆媽媽的訊息
- ☐ 去健身房運動

4 ITEMS REMAINING CLEAR DONE

精緻、有設計感

CHAPTER 06

CLAUDE.md

讓 Claude 記住你的專案

CLAUDE.md 是什麼

每次 session 啟動都會 **自動讀取**

專案知識入口：架構、技術棧、慣例、地雷

`/init` 有現成 codebase 時自動掃描生成

直接請 Claude 寫，空專案或新需求更快

有寫 vs 沒寫

輸出品質差很多

專案結構

app.py Flask API 入口

dashboard.html 前端儀表板

技術棧

- Python + Flask

- uv 管理套件

慣例

- 函式要有 type hint

- 密鑰用環境變數

地雷 (gotchas)

- dashboard.html 的 id 不能改

- Flask 跑在 port 5000

CHAPTER 07

SubAgents

一個人，多個大腦同時跑

SECTION 07 • SUBAGENTS

為什麼需要 SubAgent

每個 SubAgent 有

獨立的 context window

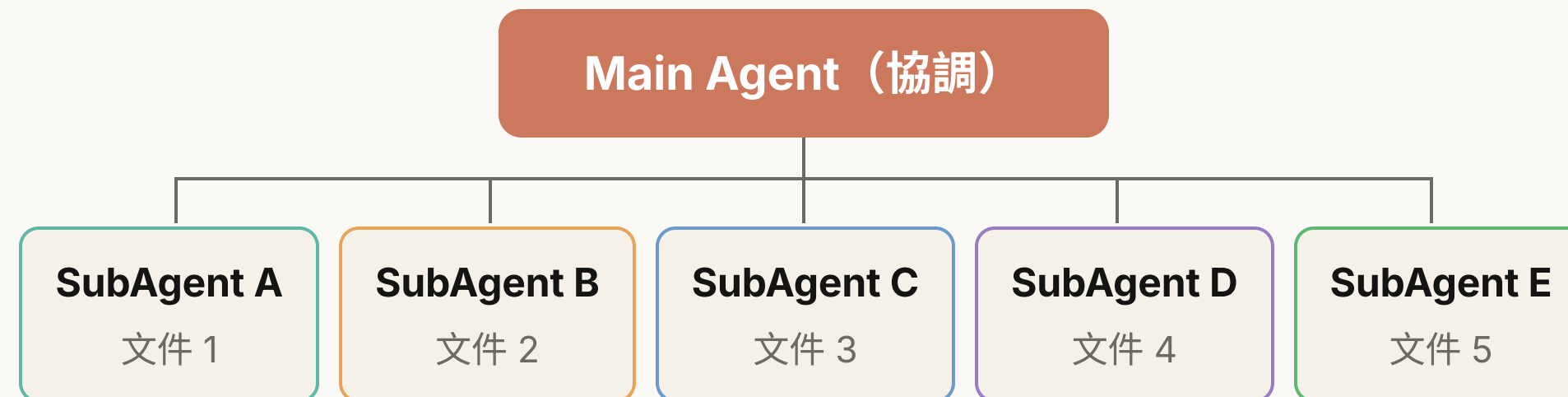
✓ 主 Agent 保持乾淨
不被探索任務汙染

✓ 多個 SubAgent **平行執行**
節省時間

```
prompt-SubAgent 對比

# 單 Agent (sequential)
幫我把 @meeting-notes/ 翻成繁中

# SubAgent 平行處理
平行處理把 @meeting-notes/ 翻成繁中
```



人格 Review SubAgents



executive

VP 層主管 · 15 年高層

技術更新轉 executive summary



user-researcher

UX 研究員 · 8 年質性

分析訪談、整理痛點



engineer

資深工程師 · Google/Meta

審查 PRD 技術可行性

prompt-人格 Review

同時用 executive、user-researcher、engineer 三個 subagent 平行處理 @prds/Carls-PRD.md 結合三者專頁的視角給出一份完整的報告

更多資源



Awesome Claude Code SubAgents

github.com/VoltAgent/awesome-claude-code-subagents

CHAPTER 08

好用 Skills 介紹

Matt Pocock Skills & OpenSpec

SECTION 08 • MATT POCKOCK

Matt Pocock Skills

github.com/mattpocock/skills

```
terminal-安裝 Skills

$ npx skills@latest add mattpocock/skills # 安裝 Matt Pocock Skills 包
```

★ **/grill-with-docs** 用 10~20 個問題逼問、釐清需求

★ **/improve-codebase-architecture** 重構、優化專案結構

★ **/tdd** 測試驅動開發流程

/to-prd 把模糊需求拆成 PRD（大問題）

/to-issues 把 PRD 拆成可執行的 issue（小問題）

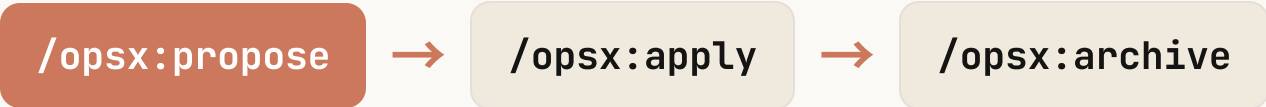
/prototype 快速建立原型驗證想法可不可行

/diagnose 照固定流程揪出難纏 bug 與變慢問題

OpenSpec : Spec-Driven Development

SDD = 規格驅動開發

寫 code 前先和 AI 對「要做什麼」達成共識。spec 是真相來源，code 跟著 spec 走。



```
terminal-安裝 OpenSpec
$ npm i -g @fission-ai/openspec@latest # 全域安裝
OpenSpec CLI
$ openspec init # 專案內初始化
```

openspec init 後的結構

- specs/ – 真相來源
 - <domain>/spec.md
- changes/ – 提案中的修改
 - <change>/
 - proposal · design · tasks
- config.yaml – 專案設定（可選）



OpenSpec 採用三階段規格驅動流程：先提出變更提案（Proposal），再設計實作方案（Design），最後拆解執行任務（Tasks）

CHAPTER 09

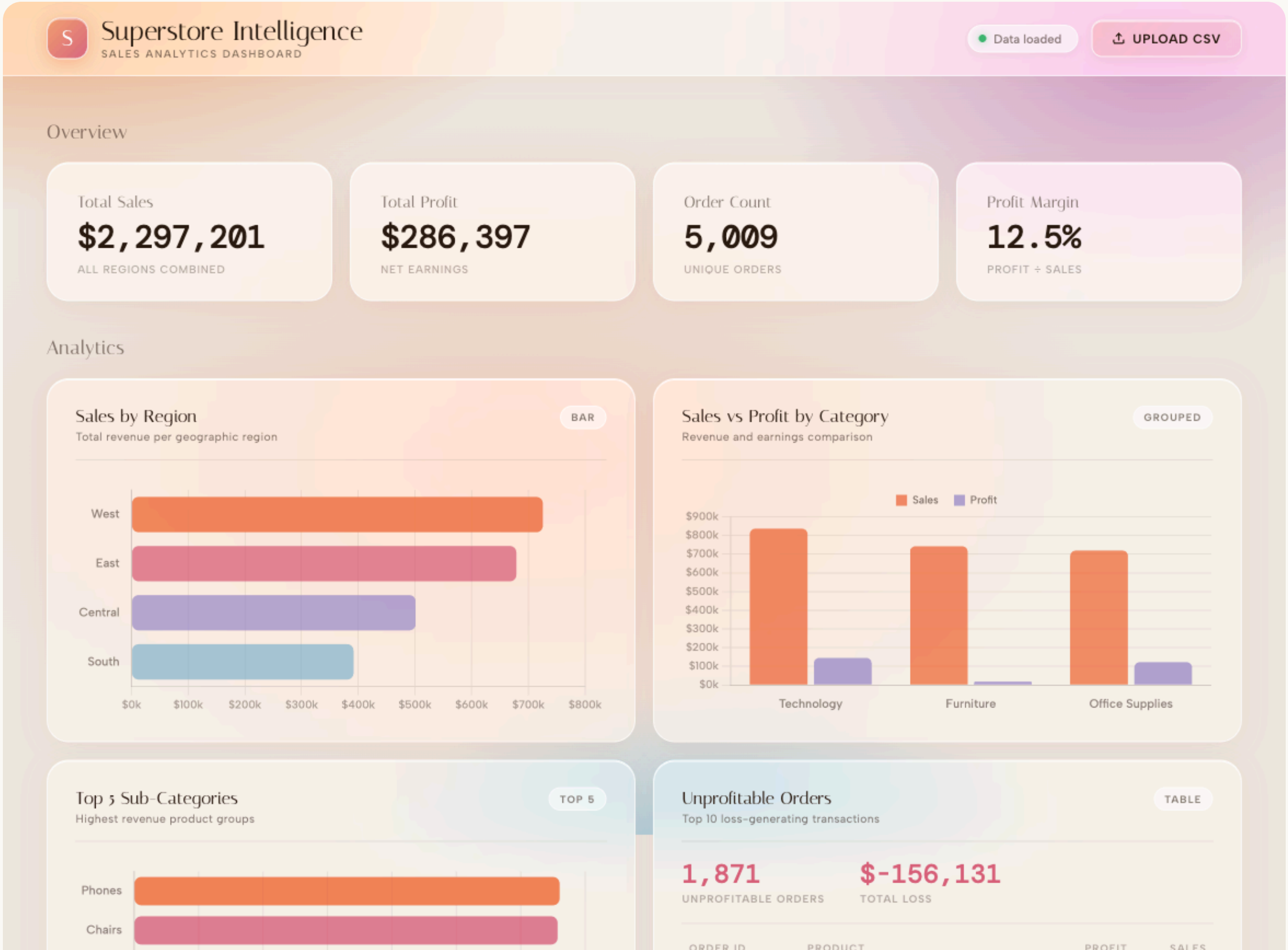
實作一個 Flask API 應用

需求確認 vs 需求模糊，兩種流程

① Flask API

② Dashboard

Dashboard 成果



Flask API

情境

需求已確認，直接進 workflow

POST /data/upload

→ 上傳 CSV，回整體摘要

GET /sales/by-region

→ 各地區銷售額排名

GET /sales/by-category

→ 各類別銷售 + 利潤對比

GET /subcategories/top?n=5

→ 銷售前 N 名子類別

GET /orders/unprofitable

→ 虧損訂單分析 (Profit < 0)

/opsx:propose → 產出 design + tasks



/opsx:apply → 依 tasks 逐步實作



prompt-opsx:propose

/opsx:propose

使用 @data/Superstore.csv 建立一個 Flask API，
API endpoint 請統一建立在 @app.py 中。

endpoints 如下：

POST /data/upload – 上傳 CSV，回傳整體摘要
(總銷售額、總利潤、訂單數、利潤率)

GET /sales/by-region – 各地區銷售額排名

GET /sales/by-category – 各類別銷售額與利潤對比

GET /subcategories/top?n=5 – 銷售前 N 名子類別

GET /orders/unprofitable – 虧損訂單分析 (Profit < 0)

套件管理使用 uv，所有回傳格式統一為 JSON。

Superstore Dashboard

情境需求模糊，先釐清再做

`/grill-with-docs` → 釐清要展示哪些資訊與圖表



`/opsx:propose` → 產出 design + tasks



`/opsx:apply` → 依 tasks 逐步實作



【可選】`/frontend-design` → 生成 dashboard.html

Superstore.csv · 9,994 筆 Kaggle 零售資料

- KPI Cards — 總覽指標
- 地區銷售 Bar chart
- 類別 Sales vs Profit 對比
- Sub-Category 排行（含負值）
- Segment Donut 佔比



prompt-opsx:apply

用 `/opsx:propose` 的 task 要使用 `/tdd` 規劃架構完成
`@dashboard.html`

可選：



prompt-frontend-design

`/frontend-design` 美化目前的 `@dashboard.html`

Statusline：隨時掌握狀態

```
● ● ● claude

• Read(src/app.py) · 讀取原始碼
• Edit(src/app.py) · 新增路由邏輯
• Bash(python app.py) · 測試啟動
> 跑一個任務中 ... |

claude-sonnet-4-6 200K v1.5.2 | course-slide (main) | +84 -12 lines
●●●●●●●●●●●●●●●●●●●● 38% | 5h 42% (1h 30m) | 7d 18% | cache 82% | 2m 30s
```

↑ 底部這條就是 *statusline*

 顯示資訊

context 用量 · 目前專案 + 分支
token 統計 · 本次 session 費用

 安裝方式

讓 Claude Code 直接讀取 `@script/README.md`
Claude Code 會根據指引進行腳本安裝

CHAPTER 10

總結

統整一下今天學到什麼

WRAP - UP

今日總結



打好地基

WSL 環境 · 四種操作模式 · 特殊符號 / ! @ · .claude/ + CLAUDE.md



掌握工具

常用指令與上下文管理 · SubAgents · Skills 安裝



實戰演練

兩個 Demo : opsx 工作流 vs grill → frontend-design

謝謝！

Q & A — 有問題歡迎提出

code.claude.com/docs/zh-TW · github.com/mattpocock/skills · [awesome-claude-code-subagents](https://github.com/mattpocock/awesome-claude-code-subagents)